

Python Data Structures: Intermediate Reference Guide

Comprehensive breakdown of method signatures, behaviors, and code examples for Lists, Tuples, Sets, and Dictionaries.

1. LISTS (ORDERED, MUTABLE, DUPLICATES ALLOWED)

Method Signature	Description	Code Example & Output
<code>append(x)</code>	Adds element <code>x</code> to the end of the list.	<code>[1, 2].append(3) → [1, 2, 3]</code>
<code>extend(iterable)</code>	Appends all elements from iterable to list.	<code>[1].extend([2, 3]) → [1, 2, 3]</code>
<code>insert(i, x)</code>	Inserts element <code>x</code> at specified index <code>i</code> .	<code>[1, 3].insert(1, 2) → [1, 2, 3]</code>
<code>remove(x)</code>	Removes first occurrence of <code>x</code> ; raises <code>ValueError</code> if absent.	<code>[1, 2, 1].remove(1) → [2, 1]</code>
<code>pop([i])</code>	Removes and returns item at index <code>i</code> (default: last).	<code>[1, 2, 3].pop() → returns 3</code>
<code>clear()</code>	Removes all items from the list in-place.	<code>lst.clear() → []</code>
<code>index(x, [start, end])</code>	Returns zero-based index of first item equal to <code>x</code> .	<code>['a', 'b'].index('b') → 1</code>
<code>count(x)</code>	Returns the number of times <code>x</code> appears in list.	<code>[1, 2, 1, 1].count(1) → 3</code>
<code>sort(key=None, reverse=F)</code>	Sorts list items in-place (modifies original).	<code>[3, 1, 2].sort() → [1, 2, 3]</code>
<code>reverse()</code>	Reverses the elements of the list in-place.	<code>[1, 2, 3].reverse() → [3, 2, 1]</code>
<code>copy()</code>	Returns a shallow copy of the list.	<code>c = lst.copy()</code>

2. TUPLES (ORDERED, IMMUTABLE, DUPLICATES ALLOWED)

Method Signature	Description	Code Example & Output
<code>count(x)</code>	Returns the number of occurrences of <code>x</code> in the tuple.	<code>(1, 2, 1, 3).count(1) → 2</code>
<code>index(x, [start, end])</code>	Returns index of first match for <code>x</code> ; raises <code>ValueError</code> if missing.	<code>('a', 'b', 'c').index('b') → 1</code>

3. SETS (UNORDERED, MUTABLE, UNIQUE ELEMENTS ONLY)

Method Signature	Description	Code Example & Output
<code>add(x)</code>	Adds element <code>x</code> to the set if not already present.	<code>{1, 2}.add(3) → {1, 2, 3}</code>
<code>remove(x)</code>	Removes element <code>x</code> ; raises <code>KeyError</code> if <code>x</code> is not found.	<code>{1, 2}.remove(1) → {2}</code>
<code>discard(x)</code>	Removes element <code>x</code> if present (no error if absent).	<code>{1, 2}.discard(3) → {1, 2}</code>
<code>pop()</code>	Removes and returns an arbitrary element; raises <code>KeyError</code> if empty.	<code>{1, 2}.pop() → returns element</code>
<code>clear()</code>	Removes all elements from the set.	<code>s.clear() → set()</code>
<code>union(*others) / </code>		

Method Signature	Description	Code Example & Output
	Returns new set with elements from set and all others.	<code>{1} {2} → {1, 2}</code>
<code>intersection(*others) / &</code>	Returns new set with elements common to all sets.	<code>{1, 2} & {2, 3} → {2}</code>
<code>difference(*others) / -</code>	Returns new set with elements in set but not in others.	<code>{1, 2} - {2, 3} → {1}</code>
<code>symmetric_difference(s) / ^</code>	Returns elements in either set but not in both.	<code>{1, 2} ^ {2, 3} → {1, 3}</code>
<code>update(*others) / =</code>	Updates set in-place with union of itself and others.	<code>s.update([2, 3])</code>
<code>issubset(other) / <=</code>	Checks if all elements of set are in other.	<code>{1}.issubset({1, 2}) → True</code>
<code>issuperset(other) / >=</code>	Checks if set contains all elements of other.	<code>{1, 2}.issuperset({1}) → True</code>
<code>isdisjoint(other)</code>	Returns True if set has no elements in common with other.	<code>{1}.isdisjoint({2}) → True</code>

4. DICTIONARIES (KEY-VALUE PAIRS, MUTABLE, UNIQUE KEYS)

Method Signature	Description	Code Example & Output
<code>get(key, [default])</code>	Returns value for key; returns default (or None) if missing.	<code>{'a': 1}.get('b', 0) → 0</code>
<code>keys()</code>	Returns a dynamic view object of dictionary keys.	<code>d.keys() → dict_keys(['a'])</code>
<code>values()</code>	Returns a dynamic view object of dictionary values.	<code>d.values() → dict_values([1])</code>
<code>items()</code>	Returns dynamic view of (key, value) tuple pairs.	<code>d.items() → dict_items([('a', 1)])</code>
<code>update([other])</code>	Updates dict with key-value pairs from other or kwargs.	<code>d.update({'b': 2})</code>
<code>pop(key, [default])</code>	Removes key and returns its value; raises KeyError if missing without default.	<code>d.pop('a') → 1</code>
<code>popitem()</code>	Removes and returns last inserted (key, value) pair.	<code>d.popitem() → ('a', 1)</code>
<code>setdefault(key, [default])</code>	Returns value if key exists; else inserts key with default and returns default.	<code>d.setdefault('k', 5) → 5</code>
<code>fromkeys(seq, [v])</code>	Creates new dict with keys from sequence and values set to v.	<code>dict.fromkeys(['a', 'b'], 0) → {'a': 0, 'b': 0}</code>
<code>clear()</code>	Removes all items from dictionary in-place.	<code>d.clear() → {}</code>

5. INTERMEDIATE COMPREHENSIVE PYTHON DEMONSTRATION CODE

```
# =====
# COMPREHENSIVE INTERMEDIATE DEMONSTRATION: LISTS, TUPLES, SETS & DICTIONARIES
# =====
# 1. LIST OPERATIONS & ADVANCED COMPREHENSION
```

```

raw_scores = [88, 92, 79, 95, 85, 100, 67]
raw_scores.sort(reverse=True)
top_scores = [score for score in raw_scores if score >= 85]
print(f"Top Scores Sorted: {top_scores}") # [100, 95, 92, 88, 85]

# 2. TUPLE UNPACKING & IMMUTABILITY
point_3d = (10, 20, 30)
x, y, z = point_3d # Unpacking
print(f"3D Coordinates: x={x}, y={y}, z={z}")

# 3. SET SET-THEORETIC OPERATIONS
group_a = {"Alice", "Bob", "Charlie", "David"}
group_b = {"Charlie", "David", "Eve", "Frank"}

common_members = group_a & group_b # Intersection
all_members = group_a | group_b # Union
unique_to_a = group_a - group_b # Difference

print(f"Common: {common_members}") # {'Charlie', 'David'}
print(f"Unique to A: {unique_to_a}") # {'Alice', 'Bob'}

# 4. DICTIONARY MANIPULATION & COMPREHENSION
student_ids = [101, 102, 103]
student_names = ["Alice", "Bob", "Charlie"]

# Zip into dictionary
student_db = dict(zip(student_ids, student_names))
student_db.setdefault(104, "David")

# Dictionary comprehension: Filter long names
long_named_students = {sid: name.upper() for sid, name in student_db.items() if len(name) > 3}
print(f"Filtered DB: {long_named_students}") # {101: 'ALICE', 103: 'CHARLIE', 104: 'DAVID'}

```